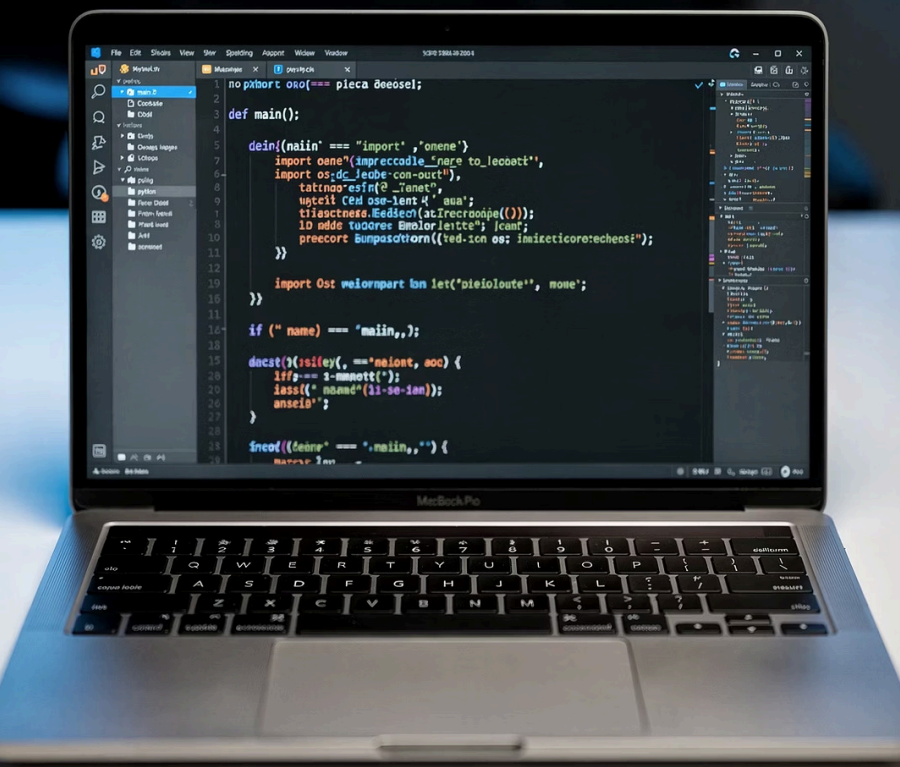




# Podstawy Pythona

Składnia, struktury danych i sterowanie przepływem — pierwsze kroki w Pythonie dla Data Science.

# Dlaczego Python?



## Popularny w data science

Python to język nr 1 w analizie danych, machine learningu i AI. Używają go m.in. Netflix, Google i NASA.



## Czytelna składnia

Kod Pythona przypomina język angielski — jest intuicyjny i łatwy do nauki nawet dla początkujących.



## Ogromny ekosystem

Biblioteki takie jak pandas, NumPy czy scikit-learn sprawiają, że Python jest gotowy do pracy z danymi od razu.

# Podstawowa składnia Pythona

## Zmienne i typy danych

```
x = 5      # int
y = 3.14   # float
z = True   # bool
name = "Anna" # str

print(type(x)) # <class 'int'>
print(type(name)) # <class 'str'>
```

- Nie deklarujemy typów — Python sam je rozpoznaje
- Komentarze zaczynają się od #
- `print()` wyświetla wartości na ekranie

# Listy — uporządkowane kolekcje danych

## Tworzenie i indeksowanie

```
data = [5, True, "Hello", 3.14]

print(data[0]) # 5
print(data[-1]) # 3.14 (ostatni)
print(data[1:3]) # [True, 'Hello']
```

## Modyfikowanie listy

```
fruits = ["jabłko", "banan"]
fruits.append("gruszka")
fruits.remove("banan")
print(fruits) # ['jabłko', 'gruszka']
```

## Przydatne metody list

- `.append(x)` — dodaje element na końcu
- `.remove(x)` — usuwa pierwsze wystąpienie
- `.count(x)` — liczy wystąpienia wartości
- `.index(x)` — zwraca pozycję elementu
- `.reverse()` — odwraca listę w miejscu
- `.sort()` — sortuje listę rosnąco

Dokumentację wszystkich metod znajdziesz wpisując `print(help(list))`.

# Słowniki — dane z kluczem

Słownik (dict) przechowuje pary **klucz: wartość**. Idealny do reprezentowania obiektów ze świata rzeczywistego.

## Tworzenie i dostęp

```
student = {  
    "imie": "Anna",  
    "wiek": 28,  
    "kierunek": "Data Science"  
}  
  
print(student["imie"]) # Anna  
print(student["wiek"]) # 28  
  
# Dodawanie nowego klucza  
student["ocena"] = 5.0  
print(student)
```

## Przydatne metody słowników

```
print(student.keys())  
# dict_keys(['imie', 'wiek', ...])  
  
print(student.values())  
# dict_values(['Anna', 28, ...])  
  
print(student.items())  
# [('imie', 'Anna'), ...]  
  
# Bezpieczny dostęp  
print(student.get("email", "brak"))  
# brak
```

# Zbiory — unikalne wartości

## Czym jest zbiór?

Zbiór (set) to kolekcja **unikalnych** elementów — duplikaty są automatycznie usuwane. Kolejność nie jest gwarantowana.

```
tags = {"python", "data", "python", "AI"}
```

```
print(tags)
```

```
# {'python', 'data', 'AI'}
```

```
tags.add("ML")
```

```
tags.discard("data")
```

```
print(tags) # {'python', 'AI', 'ML'}
```

## Operacje na zbiorach

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}
```

```
# Suma (unia)
```

```
print(A | B) # {1,2,3,4,5,6}
```

```
# Część wspólna
```

```
print(A & B) # {3, 4}
```

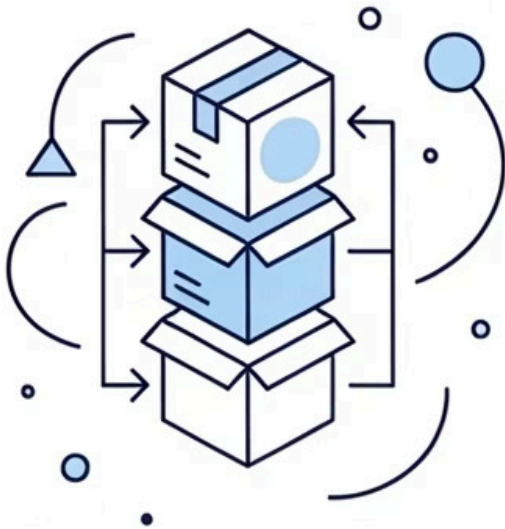
```
# Różnica
```

```
print(A - B) # {1, 2}
```

❏ Zbiory świetnie nadają się do usuwania duplikatów z danych:

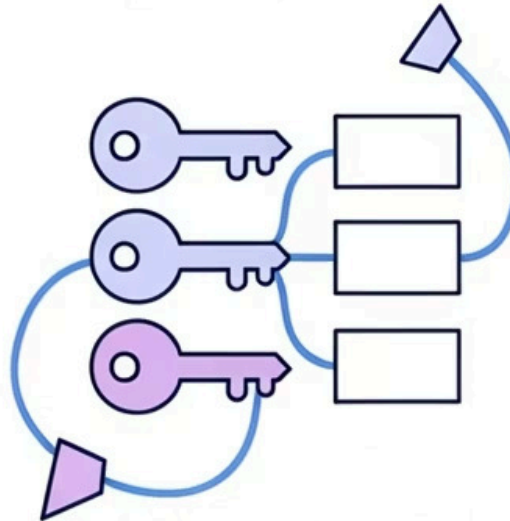
```
list(set(moja_lista))
```

# Porównanie struktur danych



## Lista (list)

Uporządkowana  
Indeksowana  
Dopuszcza duplikaty  
Przykł.: [1, 2, 3]



## Słownik (dict)

Pary klucz-wartość  
Szybki dostęp po kluczu  
Przykł.: {'imie': 'Anna'}



## Zbiór (set)

Nieuporządkowany  
Tylko unikalne elementy  
Operacje matematyczne  
Przykł.: {1, 2, 3}



# Instrukcje warunkowe: if, elif, else

Instrukcje warunkowe pozwalają programowi podejmować decyzje. Sprawdzamy warunek — jeśli jest prawdziwy (True), wykonujemy dany blok kodu.

1

if

Sprawdź pierwszy warunek

2

elif

Jeśli poprzedni fałszywy — sprawdź kolejny

3

else

Jeśli żaden nie pasuje — wykonaj domyślnie

# Przykład: Klasyfikacja temperatury powietrza

Wyobraź sobie system, który automatycznie odczytuje temperaturę powietrza i przypisuje do niej odpowiedni opis.

## Kod

```
temperature = 18 # temperatura w °C

if temperature < 0:
    description = "Mróz"
elif temperature <= 15:
    description = "Zimno"
elif temperature <= 25:
    description = "Przyjemnie"
else:
    description = "Gorąco"

print(f"Temperatura: {temperature}°C → Opis: {description}")
# Temperatura: 18°C → Opis: Przyjemnie
```

## Jak to działa?

- Python sprawdza warunki **od góry do dołu**
- Zatrzymuje się przy **pierwszym prawdziwym** warunku
- **elif** pozwala sprawdzić kolejne zakresy temperatur
- **else** to „siatka bezpieczeństwa” — wykonana się, gdy temperatura nie pasuje do żadnego wcześniejszego przedziału

📄 Zmień `temperature` na -5, 10, 22 lub 30 i sprawdź, która gałąź się wykona!

# Pętla for — iteracja po danych

Pętla for pozwala wykonać operację dla każdego elementu kolekcji. W data science używamy jej do przetwarzania rekordów, list cech czy wyników modeli.

## Iteracja po liście

```
grades = [85, 92, 67, 78, 95]

for g in grades:
    if g >= 90:
        print(f"{g} → Bardzo dobry")
    elif g >= 75:
        print(f"{g} → Dobry")
    else:
        print(f"{g} → Do poprawy")
```

```
# 85 → Dobry
# 92 → Bardzo dobry
# 67 → Do poprawy ...
```

## Iteracja po słowniku

```
student = {
    "Anna": 92,
    "Bartek": 67,
    "Celina": 85
}

for name, score in student.items():
    if score >= 80:
        status = "✓ Zaliczone"
    else:
        status = "✗ Poprawka"
    print(f"{name}: {status}")
```

# Funkcje — wielokrotne użycie kodu

Funkcja to nazwany blok kodu, który możemy wywołać wielokrotnie. Definiujemy ją słowem kluczowym `def`.

## Definicja i wywołanie

```
def calc_square(num):  
    return num * num  
  
print(calc_square(3)) # 9  
print(calc_square(5)) # 25
```

## Funkcja z kilkoma parametrami

```
def greet(name, age):  
    return f"Cześć {name}, masz {age} lat!"  
  
print(greet("Anna", 28))  
# Cześć Anna, masz 28 lat!
```

📌 Funkcja może przyjmować wiele argumentów i zwracać wynik za pomocą `return`. Bez `return` zwraca `None`.

# Metody — funkcje przypisane do obiektów

Metoda to funkcja, która **należy do konkretnego typu danych** i jest wywoływana składnią `obiekt.nazwa_metody()`.

## Metody łańcuchów (str)

```
text = 'Hello, World!'

print(text.upper())    # HELLO, WORLD!
print(text.lower())    # hello, world!
print(text.replace('World', 'Python'))
                        # Hello, Python!
print(text.split(','))
                        # ['Hello', ' World!']
print(text.startswith('Hello')) # True
print(text.strip())    # usuwa białe znaki
```

Pełna lista: `print(help(str))`

## Funkcja vs. Metoda

```
# Funkcja — niezależna, ogólna
print(sum([1, 2, 3])) # 6
print(len("hello"))  # 5
```

```
# Metoda — przypisana do obiektu
print("hello".upper()) # HELLO
nums = [3, 1, 2]
nums.sort()
print(nums) # [1, 2, 3]
```

❏ Metoda działa **tylko** na swoim typie — `.upper()` nie zadziała na liczbie!

# Podsumowanie — co dziś poznaliśmy

1

## Składnia Pythona

Zmienne, typy danych (int, float, str, bool), funkcje z def i return.

2

## Struktury danych

Lista (uporządkowana), słownik (klucz-wartość), zbiór (unikalne elementy) — każda do innych zastosowań.

3

## Warunki if / elif / else

Podejmowanie decyzji w kodzie — sprawdzanie warunków od góry, zatrzymanie przy pierwszym prawdziwym.

4

## Pętle i metody

Pętla for do iteracji po danych; metody jako funkcje przypisane do obiektów (obiekt.metoda()).